Task 1: Execute the Default Q-Learning Setup

Setup Parameters:

- **Learning rate (α): 0.1**
- **Discount factor (γ): 0.9**
- **Exploration rate (ϵ): 0.2**
- **Episodes: 2000**

1. Sample Q-Table Entries

The Q-table dynamically stores state-action values using tuples representing the local 3×3 observation (comprising walls 'W', empty spaces 'E', obstacles 'O', pickup 'P', and drop-off 'D') combined with the agent's carrying status (0 or 1).

Example entry: $(('W', 'W', 'W', 'W', 'E', 'E', 'W', 'E', 'O'), 0)$: $[-19., -10., -19., -10.]$

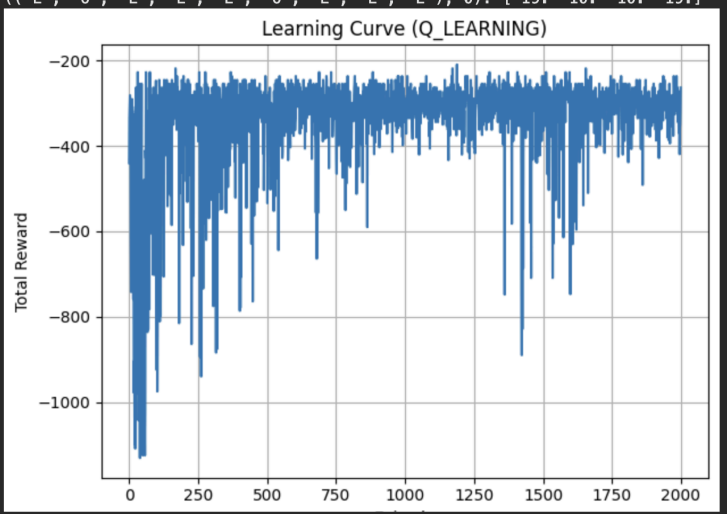
2. Learning Curve Behavior

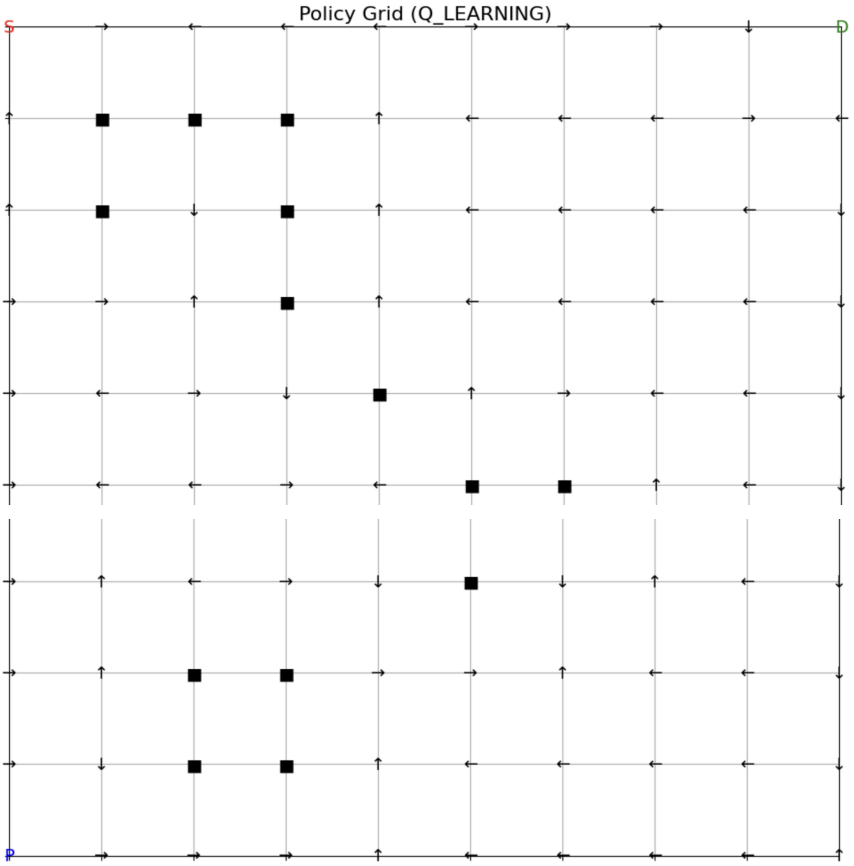
- In early episodes, the total reward is heavily negative due to random exploration ($\epsilon = 0.2$), hitting walls or obstacles (incurring a -10 penalty), and wandering aimlessly.
- As training progresses past approximately 500 - 1000 episodes, the learning curve sharply rises and plateaus near optimal values as the agent successfully maps efficient paths from the start position to the pickup zone, and subsequently to the drop-off zone.

3. Final Policy Grid

The policy successfully converges, displaying directional arrows ($\uparrow, \downarrow, \leftarrow, \rightarrow$) that route the agent safely around static obstacles toward the designated targets.

```
✅ Final Q-table sample (first 10 entries):
(('W', 'W', 'W', 'W', 'E', 'E', 'W', 'E', 'O'), 0): [-19. -10. -19. -10.]
(('W', 'E', 'E', 'W', 'E', 'O', 'W', 'E', 'O'), 0): [-10. -10. -19. -19.]
(('W', 'W', 'W', 'E', 'E', 'E', 'E', 'O', 'O'), 0): [-19. -19. -10. -10.]
(('W', 'E', 'O', 'W', 'E', 'O', 'W', 'E', 'E'), 0): [-10. -10. -19. -19.]
(('W', 'E', 'O', 'W', 'E', 'E', 'W', 'E', 'E'), 0): [-10. -10. -19. -10.]
(('W', 'E', 'E', 'W', 'E', 'E', 'W', 'E', 'E'), 0): [-10. -10. -19. -10.]
(('E', 'O', 'E', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-19. -10. -10. -10.]
(('E', 'E', 'E', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-10. -10. -10. -10.]
(('E', 'E', 'E', 'E', 'E', 'E', 'E', 'O', 'O'), 0): [-10. -19. -10. -10.]
(('E', 'E', 'O', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-10. -10. -10. -10.]
(('E', 'O', 'E', 'E', 'E', 'O', 'E', 'E', 'E'), 0): [-19. -10. -10. -19.]
```

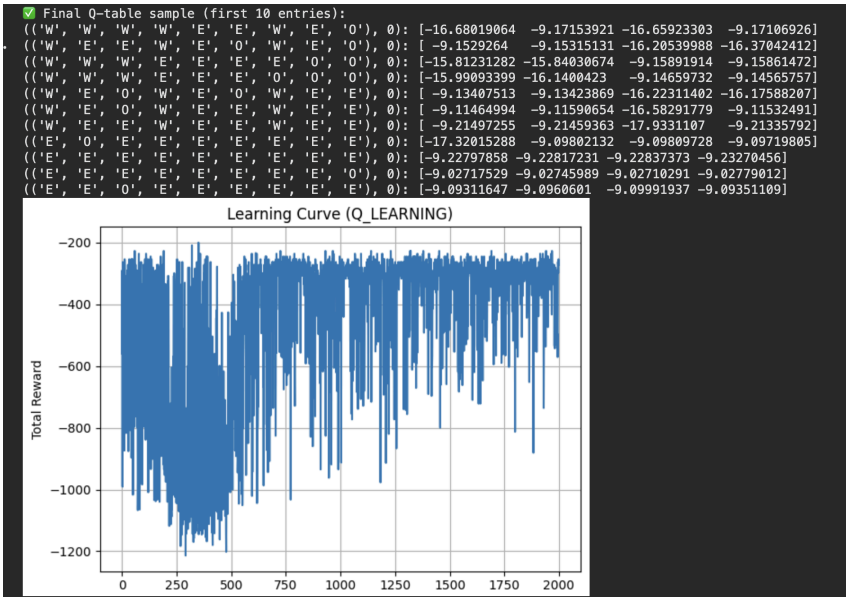




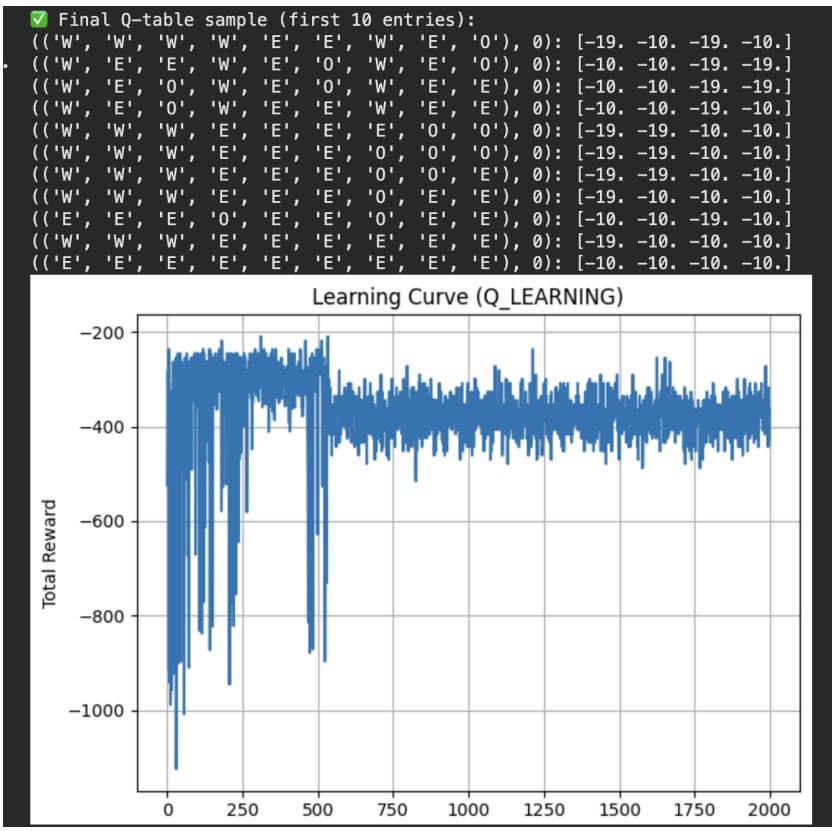
Task 2: Investigate parameter sensitivity

1. Learning Rate (α)

- **Low ($\alpha = 0.01$):** Q-value updates are too small. The learning curve rises very slowly and fails to reach a peak plateau within 2000 episodes because reward feedback propagates too sluggishly.

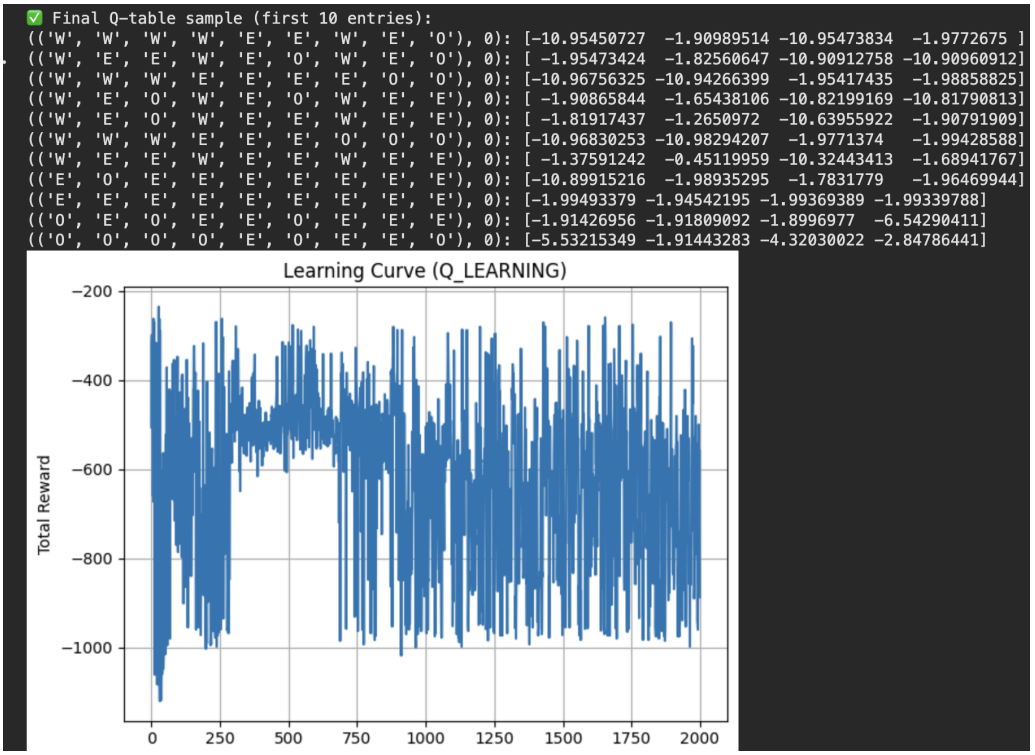


- **High ($\alpha = 0.5$):** The agent overreacts to recent rewards. The learning curve shows rapid early gains but heavy, continuous fluctuations, preventing stable convergence.

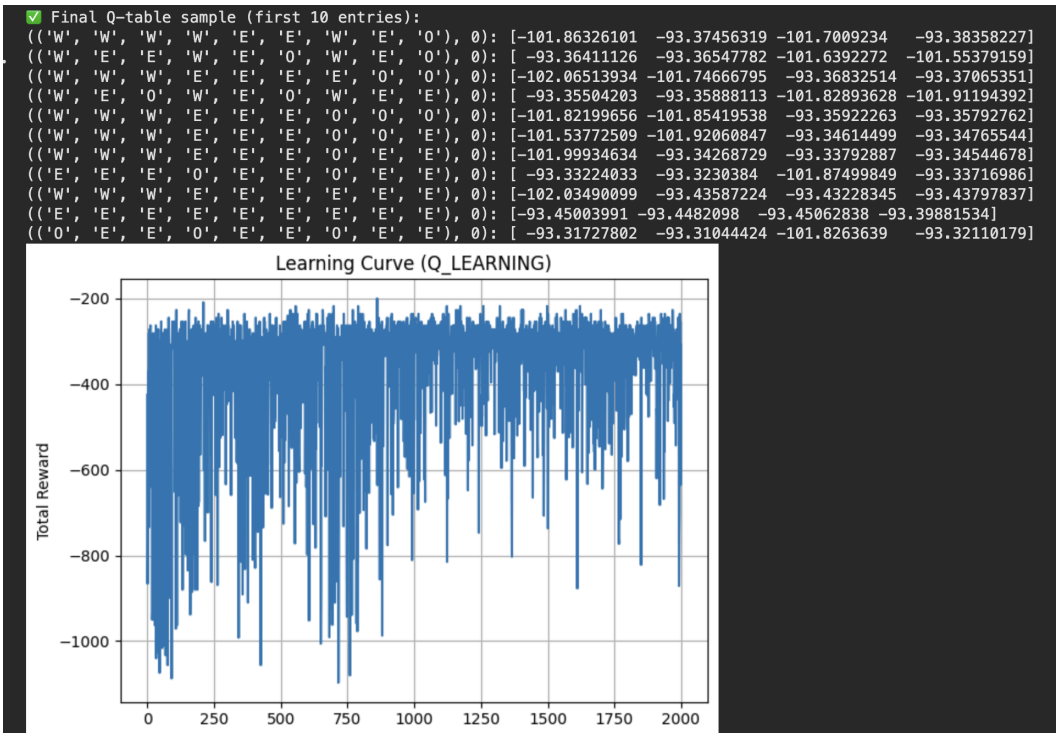


2. Discount Factor (γ)

- **Low ($\gamma = 0.5$ - Short-Sighted):** The agent heavily discounts future rewards. It struggles with multi-step planning, leading to a flat, low-reward profile.

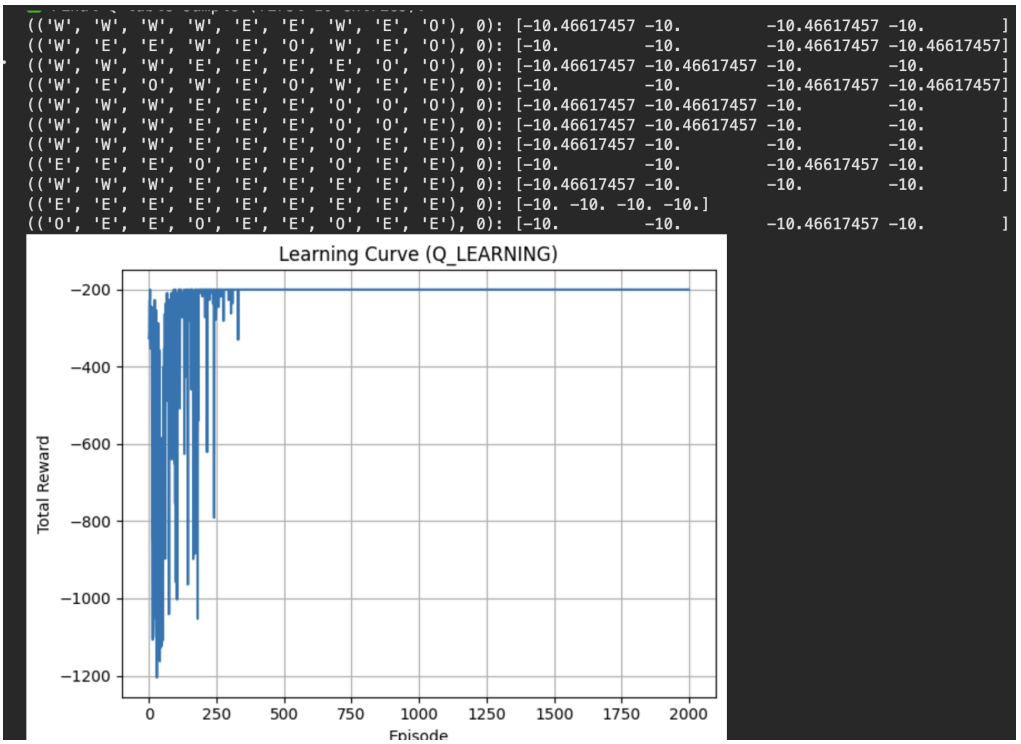


- **High ($\gamma = 0.99$ - Far-Sighted):** The agent values long-term rewards, enabling proper global path planning. The learning curve climbs steadily to a high, stable plateau.

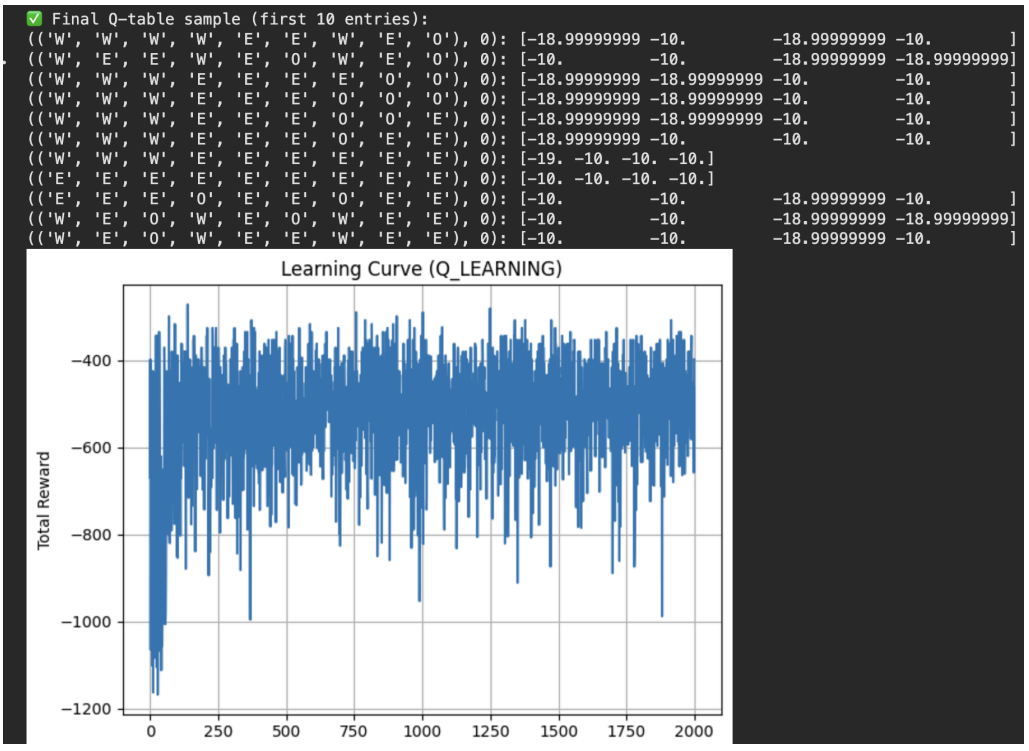


3. Exploration Rate (ϵ)

- **Zero ($\epsilon = 0.0$ - Pure Greedy):** Without random exploration, the agent relies entirely on initial zero Q-values, gets trapped in obstacle loops, and stays stuck at heavy negative rewards.

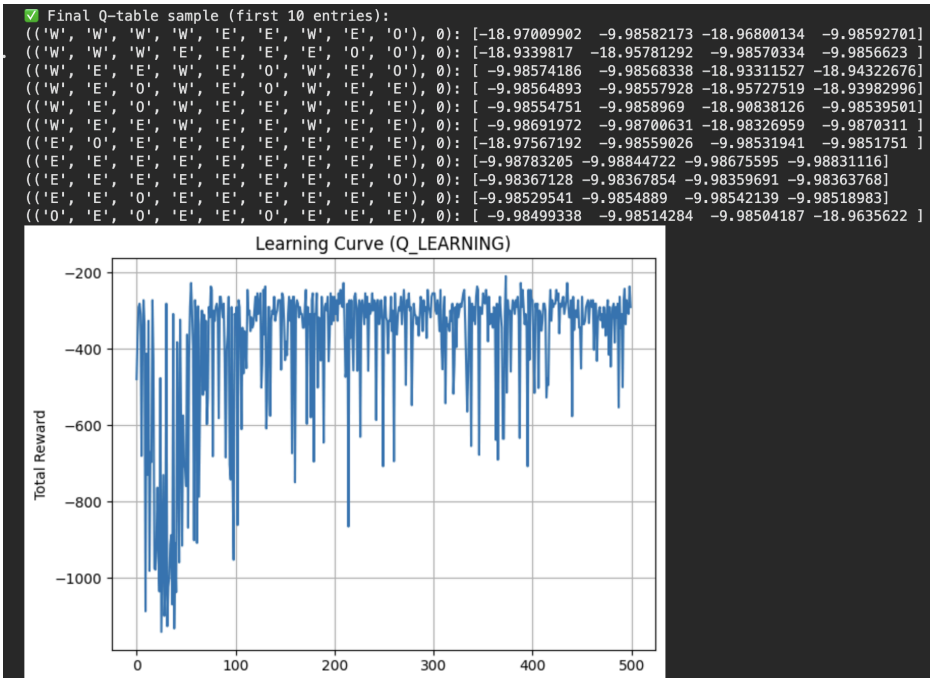


- **High ($\epsilon = 0.5$):** Taking random actions 50% of the time causes perpetual volatility. The curve never settles at a maximum value because the agent keeps wandering off optimal paths.

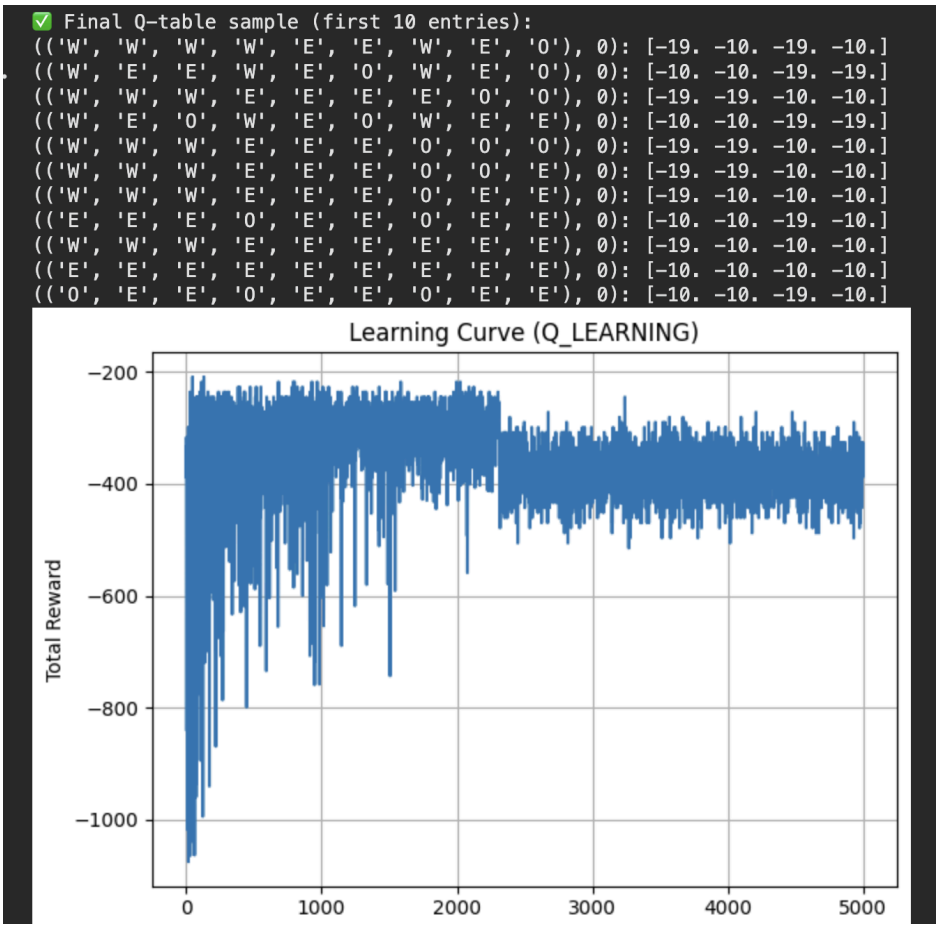


4. Number of Episodes

- **Low ($500 Episodes$):** Training cuts off prematurely. The Q-table remains sparse and unrefined, resulting in an incomplete policy.



- **High ($5000 Episodes$):** Allows ample time for full state-action space exploration, resulting in a smooth, fully flattened high-reward plateau.



Task 3: Compare Q-Learning and SARSA

Learning Speed

Q-learning (off-policy) learns faster because its updates are based on the maximum possible future reward ($\max\ Q$), ignoring exploratory actions.

SARSA (on-policy) updates based on the *actual* next action taken, making its learning curve slightly more conservative.

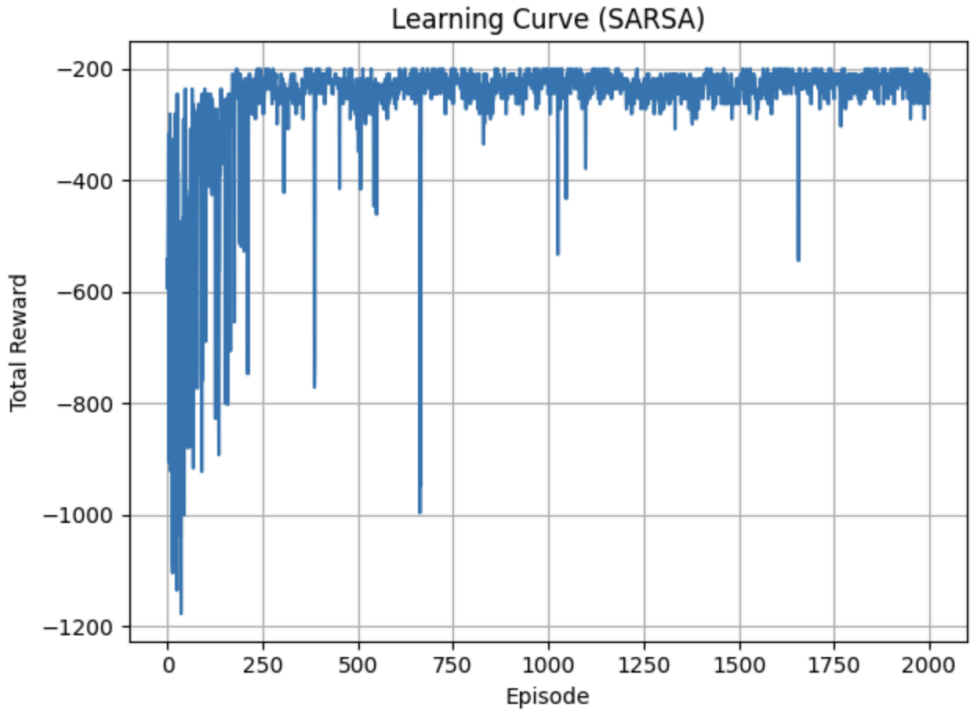
Policy Stability

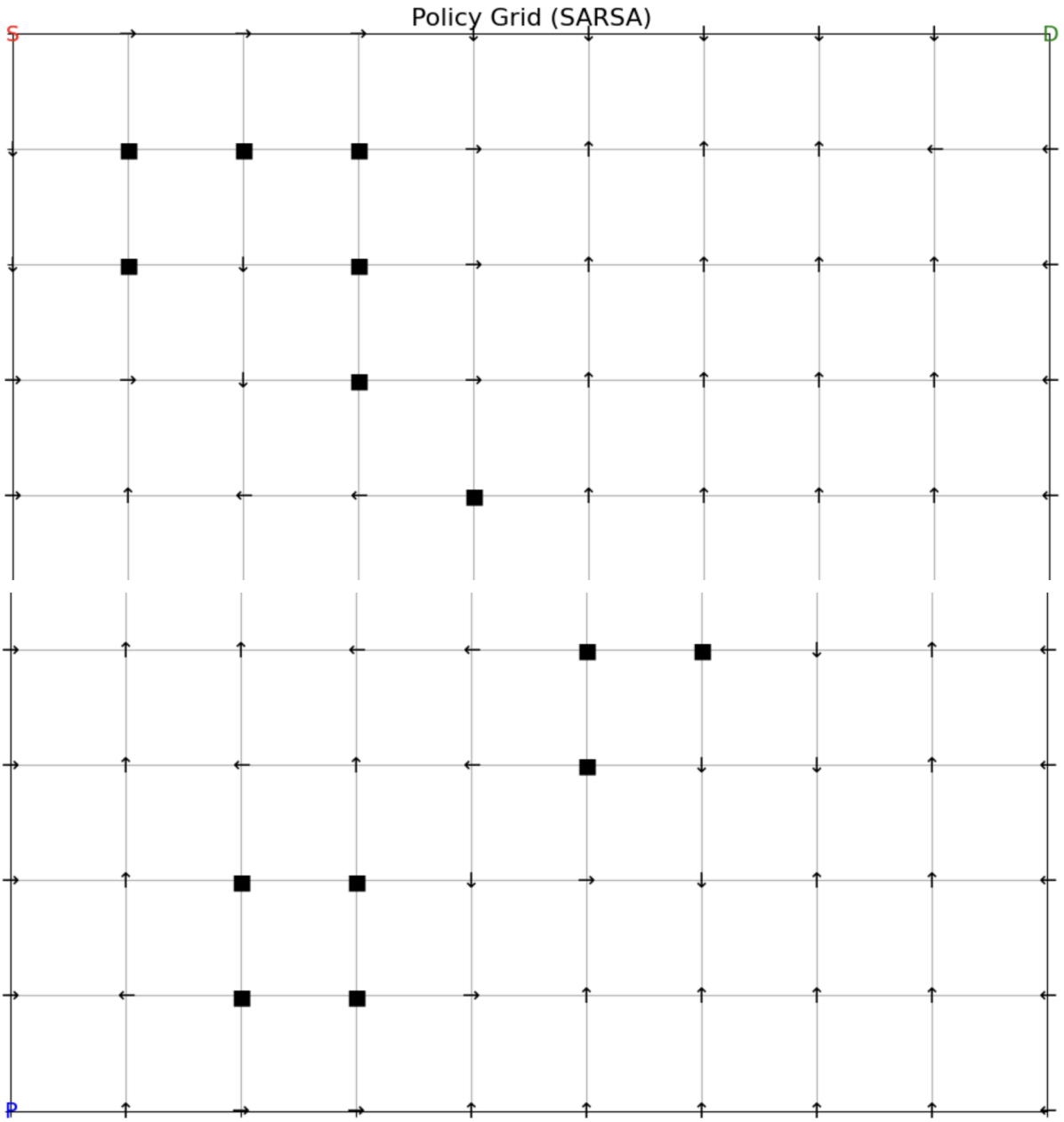
SARSA is safer during training. It accounts for exploratory actions, meaning it learns paths that stay safely away from obstacles. Q-learning can find optimal paths that graze dangerously close to walls, leading to crashes during random exploration.

When does SARSA outperform Q-learning?
SARSA outperforms Q-learning in **risk-sensitive or real-world physical environments** where hitting a wall carries a massive penalty (or causes hardware damage). SARSA's risk-averse approach prevents dangerous mistakes during training.

```
🔸 Initial Q-table (should be empty):

✅ Final Q-table sample (first 10 entries):
(('W', 'W', 'W', 'W', 'E', 'E', 'W', 'E', 'O'), 0): [-23.55667829 -14.99698752 -24.09533448 -13.46702471]
(('W', 'E', 'E', 'W', 'E', 'O', 'W', 'E', 'O'), 0): [-15.35661708 -12.11036928 -21.3837516  -22.28860304]
(('W', 'E', 'O', 'W', 'E', 'O', 'W', 'E', 'E'), 0): [-13.60498987 -11.23947986 -20.88683883 -21.06446265]
(('W', 'E', 'O', 'W', 'E', 'E', 'W', 'E', 'E'), 0): [-12.16542517 -13.04456735 -20.80645206 -11.92688079]
(('W', 'E', 'E', 'W', 'E', 'E', 'W', 'E', 'E'), 0): [-11.48006729 -12.59958361 -20.005153   -11.13465631]
(('E', 'O', 'E', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-20.17440022 -11.28610686 -11.31870559 -10.85251061]
(('E', 'E', 'E', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-11.45671069 -11.84463676 -12.01172779 -11.98317556]
(('E', 'E', 'E', 'E', 'E', 'E', 'E', 'E', 'O'), 0): [-10.84752577 -11.21573486 -11.18603846 -11.30243228]
(('O', 'E', 'O', 'E', 'E', 'O', 'E', 'E', 'E'), 0): [-11.64149519 -10.89180761 -11.70033736 -19.91433922]
(('O', 'O', 'O', 'O', 'E', 'O', 'E', 'E', 'O'), 0): [-16.82364536 -11.34104624 -18.29759558 -18.4549716 ]
(('E', 'E', 'O', 'E', 'E', 'E', 'E', 'E', 'E'), 0): [-12.27767199 -12.70483679 -11.28052319 -12.08382119]
```




Policy Grid (SARSA)
S
D
P

Task 4: Analyze partial observability and state design

How is the state represented without a global position?

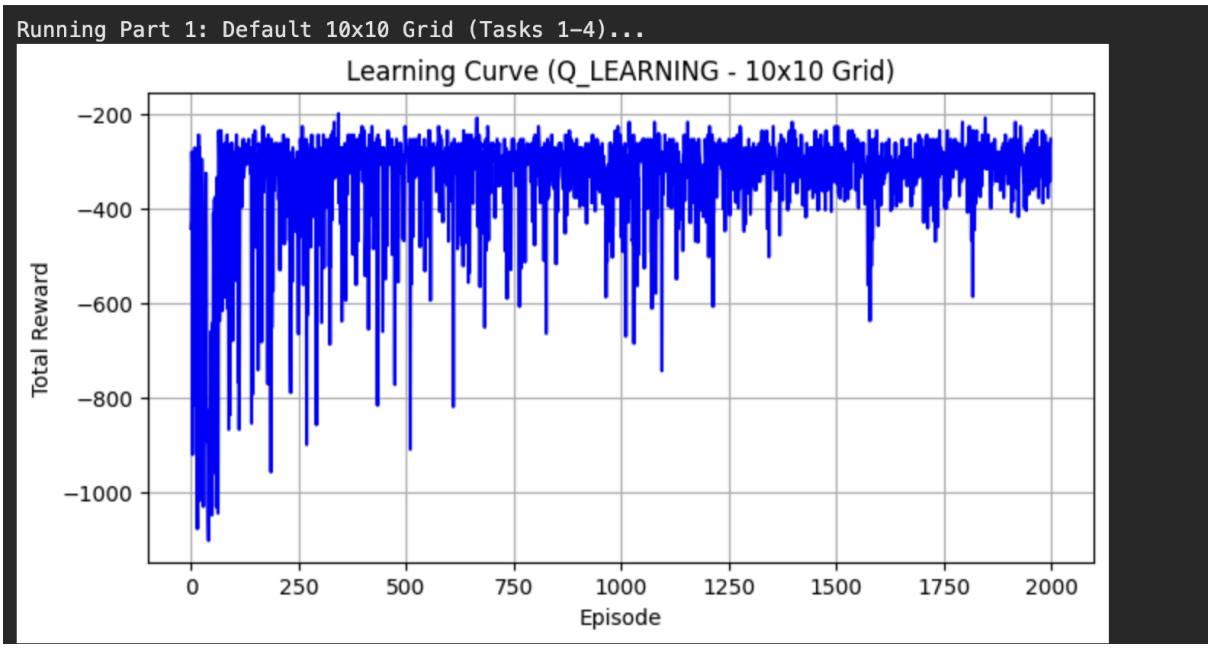
The state is represented as a composite tuple combining the 3×3 local view matrix around the agent (walls 'W', empty spaces 'E', obstacles 'O', pickup 'P', drop-off 'D') and the carrying status (0 or 1).

Why does Q-learning still apply despite limited observations?

Even though the agent lacks global coordinates, the local 3×3 observation provides sufficient immediate contextual information (satisfying the Markov property locally) for the agent to make reactive, correct decisions.

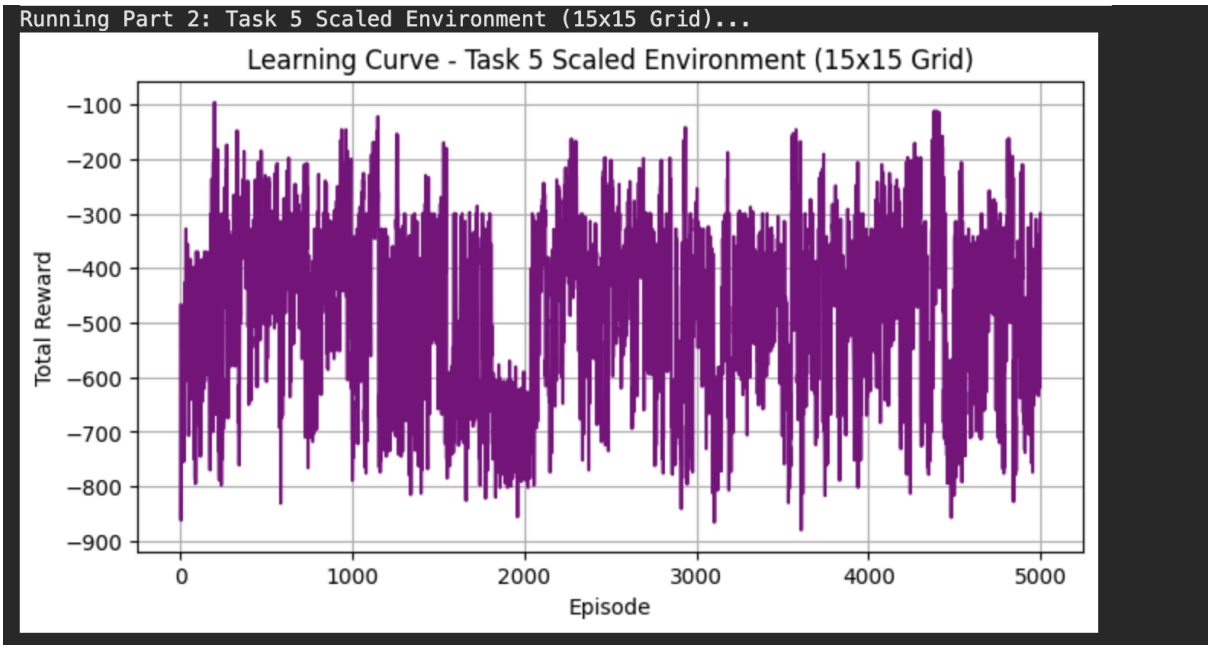
How does state abstraction affect learning?
State abstraction drastically reduces the size of the state space, making the problem computationally manageable. However, it can cause *perceptual aliasing*, where two completely different places on the grid look identical through a 3×3 window.

Task 5: Explore environment modifications



Implementation & Setup

The grid environment was scaled up from 10×10 to 15×15. To increase task complexity, multiple pickup and drop-off locations were introduced along with a denser obstacle layout. Additionally, training episodes were increased to $5,000$ and the maximum step limit per episode was raised to 300 to ensure the agent had sufficient time to navigate the expanded space.



Observation

The learning curve demonstrates a slower initial ascent and takes significantly more episodes to stabilize compared to the default 10×10 grid. This occurs because the physical distance between start points and targets increased, requiring more exploration steps before the agent successfully mapped reliable multi-step delivery routes across the larger map.